МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Построение и анализ алгоритмов»

Тема: Редакционное расстояние (Левейнштейн)

Вариант: 3а

Студент гр. 4343 _____ Едыгеев Р. Б.

Преподаватель _____ Калишенко Е. Л.

Санкт-Петербург

2026

Цель работы.

Необходимо реализовать программу, вычисляющую редакционное расстояние между двумя строками, а также добавить к существующим операциям новую – последовательная вставка двух одинаковых символов.

Задание.

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую. Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк pedestal и stien расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа: pedestal - stal.
- Затем необходимо заменить два последних символа: stal - stie.
- Потом нужно добавить символ в конец строки: stie - stien.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв (S, $1 \leq |S| \leq 2550$).

Вторая строка входных данных содержит строку из строчных латинских букв. (T, $1 \leq |T| \leq 2550$).

Параметры выходных данных:

Одно число L, равное расстоянию Левенштейна между строками S и T.

Задание варианта.

Добавляется 4-я операция со своей стоимостью: последовательная вставка двух одинаковых символов.

Основные теоретические положения.

Расстояние Левенштейна определяется как минимальное количество операций вставки, удаления и замены одного символа, необходимых для превращения одной строки в другую. В основе вычисления лежит метод динамического программирования. Строится матрица размером (m + 1) x (n + 1), где m и n длины исходной и целевой строк соответственно. Нулевая строка и нулевой столбец инициализируются значениями от 0 до m и от 0 до n, что соответствует стоимости вставки или удаления всех символов для получения пустой строки. Для каждой ячейки с индексами i и j, где i больше нуля и j больше нуля, вычисляется стоимость замены символа: если символы в позициях i - 1 первой строки и j - 1 второй строки совпадают, стоимость равна 0, иначе стоимость операции замены + число в данной позиции. Значение в ячейке определяется как минимум из трёх величин: значения ячейки слева плюс единица, что соответствует удалению символа из первой строки; значения ячейки сверху плюс единица, что соответствует вставке символа во вторую строку; и значения ячейки по диагонали слева сверху плюс стоимость замены. Итоговое расстояние Левенштейна находится в правой нижней ячейке матрицы с индексами m и n.

Для оптимизации памяти вместо полной матрицы можно хранить только две строки или два столбца, поскольку каждая ячейка зависит только от соседних уже вычисленных значений.

Для добавления операции последовательной вставки двух одинаковых символов достаточно в каждой позиции рассматривать равенство символов строки 2 в двух предыдущих позициях (j – 1, j – 2). Если они равны – то набранная до этого минимальная стоимость сравнивается с суммой числа в позиции (I, j – 2) и стоимостью операции последовательной вставки.

Выполнение работы.

Для решения задачи была разработана программа на языке Python (см. Приложение А).

Главным оптимизационным решением является хранение не всей таблицы, а лишь двух предыдущих строк и строки-столбца, вследствие чего даже для больших строк затраты по памяти будут сравнительно невелики.

Пайплайн работы алгоритма.

Программа основана на построении таблицы весов размером (m+1) x (n + 1), где m – длина второй строки, n – длина первой. Сохраняются лишь строки arr[0] и arr[1], которые в ходе заполнения таблицы обновляются.

Каждая новая строка заполняется для каждого элемента выбором минимальной стоимости суммы значений среди трех “соседей” этой клетки и стоимости операций данных клеток: arr[j - 1][i] - вставка, arr[j][I – 1] - удаление, arr[j – 1][I - 1] – замена.

Также рассматривается возможность использования последовательной вставки двух одинаковых элементов.

Строка заполняется, если строка уже заполнена, то строка last_last заменятся на last, а last на только что заполненную строку new.

В качестве ответа выводится значение самой правой нижней клетки arr[m][n] – это и будет минимальная стоимость для преобразования одной строки в другую.

Каждая итерация фиксируется в лог-файл log.txt, чтобы можно было отследить полный путь алгоритма. Также каждую заполненную строку программа записывает в файл out.txt, чтобы можно было в конце выполнения программы увидеть заполненную таблицу алгоритма.

В качестве дополнительного и отладочного вывода был реализован обратный алгоритм – редакционное предписание, который по таблице из out.txt восстанавливает операции, нужные для превращения одной строки в другую. Алгоритм учитывает наличие добавленной операции.

Анализ сложности алгоритма.

Весь алгоритм сводится к построению матрицы (m + 1)x(n + 1), следовательно сложность по времени составит O(m*n).

Так как оптимизация позволяет хранить всего 3 строки и заполнять четвертую, то сложность по памяти не составит более O(3n + m) или же O(n + m).

Тестирование.

Для тестирования были написаны следующие тесты, покрывающие граничные случаи:

1. Две пустые строки (Рис. 1):

```
crakeny@LAPTOP-TQ9SBN6I:/mnt/c/Users/Hp/OneDrive/Рабочий стол/projects/2_COURSE/BAAA/lb3/src$ python3 main.py


0
crakeny@LAPTOP-TQ9SBN6I:/mnt/c/Users/Hp/OneDrive/Рабочий стол/projects/2_COURSE/BAAA/lb3/src$
```

Рисунок 1 - Тест №1, две пустые строки

2. Две одинаковые строки (Рис. 2):

```
out.txt
hello
hello

0	1	2	3	4	5
1	0	1	2	3	4
2	1	0	1	2	3
3	2	1	0	1	2
4	2	1	1	0	1
5	3	2	2	1	0
(M - h = h)
(M - e = e)
(M - l = l)
(M - l = l)
(M - o = o)
Answer: 0
```

Рисунок 2 - Тест №2, две идентичные строки

3. Одна пустые строки (Рис. 3):

```
out.txt

hello

0
1
2
3
4
5
(I - h)
(I - e)
(I - l)
(I - l)
(I - o)
Answer: 5
```

Рисунок 3 - Тест №3, две пустые строки

4. Проверка работы двойной вставки (стоимость insert – 1, double insert – 1)(Рис. 4):

```
out.txt
cat
coot

0 1 2 3
1 0 1 2
2 1 1 2
3 1 2 2
4 2 2 2
(M - c = c)
(D - a)
(DI - o)
(M - t = t)
Answer: 2
```

Рисунок 4 - Тест №4, две идентичные строки

5. Проверка работы двойной вставки (стоимость insert – 1, double insert – 2)(Рис. 5):

```
out.txt
0 1 2 3
1 0 1 2
2 1 1 2
3 2 2 2
4 3 3 2
(M - c = c)
(R - a -> o)
(I - o)
(M - t = t)
Answer: 2
```

Рисунок 5 - Тест №5, две идентичные строки

Из вывода тестов 4 и 5 видно, что алгоритм корректно выбирает минимальную стоимость для своих действий: в первом случае две обычные вставки стоили бы 2, а одна двойная – 1; во втором же замена + вставка вышли дешевле, чем удаление + двойная вставка за 2 ус. ед.

Также приведен пример логирования выполнения программы (Рис. 6):

```
# c a t -----------Chosen operation: None
# 0 1 2 3
c 1 0 0 0

# c a t -----------Chosen operation: insert
# 0 1 2 3
c 1 0 1 0

# c a t -----------Chosen operation: insert
# 0 1 2 3
c 1 0 1 2

# c a t -----------Chosen operation: delete
# 1 0 1 2
o 2 1 0 0

# c a t -----------Chosen operation: replace
# 1 0 1 2
o 2 1 1 0

# c a t -----------Chosen operation: insert
# 1 0 1 2
o 2 1 1 2

# c a t -----------Chosen operation: delete
# 2 1 1 2
o 3 2 0 0

# c a t -----------Chosen operation: delete
# 2 1 1 2
o 3 2 2 0
```

Рисунок 6 - Логирование

Вывод.

В ходе выполнения лабораторной работы был изучен алгоритм нахождения редакционного расстояния между двумя строками. Была разработана программа на Python, выполнен анализ сложности алгоритма по времени и памяти, проведено тестирование программы.

Были реализованы оптимизация алгоритма по памяти, а также алгоритмы обратного хода алгоритма и логирование.

Был получен опыт построения алгоритма для решения задачи, а также опыт в сфере оценки и анализа проведенной работы.

ПРИЛОЖЕНИЕ А
КОД ПРОГРАММЫ

Файл main.py:

```
import os


def show(str1, str2, last, new, i, choise):
    with open("log.txt", "a") as f:
        f.writelines("  #\t")
        f.writelines(f'{i}\t' for i in str1)
        f.writelines('-----------' + f"Chosen operation: {choise}" +
'\n')

        f.writelines("# ")
        f.writelines(f'{i}\t' for i in last)
        f.write('\n')

        f.writelines(str2[i - 1] + ' ')
        f.writelines(f'{i}\t' for i in new)
        f.write('\n')
        f.write('\n')

ops = {
    "del" : 1,
    "ins" : 1,
    "rep" : 1,
    "dins" : 2
}

def reconstruction(size, str1, str2):
    arr = []
    with open("out.txt", "r") as f:
        for _ in range(size):
            row = list(map(float, f.readline().split()))
            arr.append(row)

    j = len(arr) - 1
    i = len(arr[0]) - 1
    stack = []

    while j > 0 or i > 0:
        current = arr[j][i]

        if j >= 2 and str2[j-1] == str2[j-2]:
            if abs(current - (arr[j-2][i] + ops["dins"])) < 1e-9:
                stack.append(('(', 'DI', ' - ', str2[j - 2], ')'))
                j -= 2
                continue

        if j > 0 and abs(current - (arr[j-1][i] + ops["del"])) < 1e-9:
            stack.append(('(', 'I', ' - ', str2[j - 1], ')'))
            j -= 1
            continue

        if i > 0 and abs(current - (arr[j][i-1] + ops["ins"])) < 1e-9:
            stack.append(('(', 'D', ' - ', str1[i - 1], ')'))
            i -= 1
```

```
            continue

        if j > 0 and i > 0:
            if str1[i-1] == str2[j-1]:
                if abs(current - arr[j-1][i-1]) < 1e-9:
                    stack.append(('(', 'M', ' - ', str1[i - 1], ' = ',
str2[j - 1], ')'))
                    j -= 1
                    i -= 1
                    continue
            else:
                if abs(current - (arr[j-1][i-1] + ops["rep"])) < 1e-9:
                    stack.append(('(', 'R', ' - ', str1[i - 1], ' ->
', str2[j - 1], ')'))
                    j -= 1
                    i -= 1
                    continue
        print(f"Error at ({j},{i}), current={current}")
        break
    stack.reverse()
    return stack

def main():
    if os.path.exists("log.txt"):
        os.remove("log.txt")

    if os.path.exists("out.txt"):
        os.remove("out.txt")

    str1 = list(input())
    str2 = list(input())

    if not str1:
        str1 = ""

    if not str2:
        print(len(str1))
        return

    n = len(str1) + 1
    m = len(str2) + 1

    row = [x for x in range(0, n)]
    col = [x for x in range(0, m)]

    last_last_row = row
    last_row = [0] * (n - 1)
    last_row.insert(0, col[1])
    for j in range(1, n):
        choise = None
        if str1[j - 1] == str2[0]:
            last_row[j] = min(last_last_row[j] + ops["del"],
last_row[j - 1] + ops["ins"], last_last_row[j - 1])
        elif str1[j - 1] != str2[0]:
            last_row[j] = min(last_last_row[j] + ops["del"],
last_row[j - 1] + ops["ins"], last_last_row[j - 1] + ops["rep"])
```

```
        if last_row[j] == last_last_row[j] + ops["del"]: choise =
"delete"
        elif last_row[j] == last_row[j - 1] + ops["ins"]: choise =
"insert"
        elif last_row[j] == last_row[j] == last_last_row[j - 1] +
ops["rep"]: choise = "replace"
        elif last_row[j] == 0: choise = "None"
        show(str1, str2, last_last_row, last_row, 1, choise)

    with open("out.txt", "w") as out:
        out.writelines(f'{i}\t' for i in last_last_row)
        out.write('\n')
        out.writelines(f'{i}\t' for i in last_row)
        out.write('\n')

    for i in range(2, m):
        new = [0]*(n - 1)
        new.insert(0, col[i])
        for j in range(1, n):
            choise = None
            cost = 0
            if str1[j - 1] == str2[i - 1]:
                cost = min(last_row[j] + ops["del"], new[j - 1] +
ops["ins"], last_row[j - 1])
            elif str1[j - 1] != str2[i - 1]:
                cost = min(last_row[j] + ops["del"], new[j - 1] +
ops["ins"], last_row[j - 1] + ops["rep"])

            if str2[i - 1] == str2[i - 2]:
                cost = min(cost, last_last_row[j] + ops["dins"])

            new[j] = cost
            if new[j] == last_row[j] + ops["del"]: choise = "delete"
            elif new[j] == new[j - 1] + ops["ins"]: choise = "insert"
            elif new[j] == last_row[j - 1] or new[j] == last_row[j -
1] + ops["rep"]: choise = "replace"
            elif new[j] == last_last_row[j] + ops["dins"]: choise =
"double insertion"
            elif last_row[j] == 0: choise = "None"
            show(str1, str2, last_row, new, i, choise)

        last_last_row = last_row
        last_row = new
        with open("out.txt", "a") as out:
            out.writelines(f'{i}\t' for i in last_row)
            out.write('\n')

    print(last_row[-1])
    reverse = reconstruction(m, str1, str2)
    with open("out.txt", "a") as f:
        for line in reverse:
            f.writelines(f'{i}' for i in line)
            f.write('\n')
        f.write(f"Answer: {str(last_row[-1])}")

if __name__ == "__main__":
    main()
```